

RESEARCH FUNDAMENTALS – MINOR PROJECT PROPOSAL

Building a GPT-Style Language Model from Scratch

A transparent, tested, and fully documented implementation of the GPT architecture — from byte-pair tokenization to trained text generation.

Bikram Sharma • Pramesh Lamichhane • Swikar Poudel
Gandaki College of Engineering and Science | June 2026

AGENDA

What We Will Cover

- | | | | |
|----|-------------------------------|----|------------------------------------|
| 01 | Background and Problem | 04 | System Design Diagrams |
| 02 | Objectives and Scope | 05 | Methodology and Testing |
| 03 | Repository and Tools | 06 | Risk, Timeline and Outcomes |

CHAPTER 1 · INTRODUCTION

Why Build a GPT from Scratch?

- Most students only ever see the output side of language models — a chat box.
- GPT-style models predict the next token from all previous tokens (Radford et al., 2019).
- Self-attention (Vaswani et al., 2017) replaced recurrence and made this trainable at scale.
- This project builds every component directly — tokenizer, embeddings, attention, training, generation.

Base Repository

GPT-From-scratch

- `config.py` — GPT-124M configuration
- `model/gpt.py`, `model/transformer.py`
- Training + tutorial notebooks
- Sample corpus: `the-verdict.txt`
github.com/beeks-code/GPT-From-scratch

CHAPTER 1 · INTRODUCTION

Problem Statement

“Many learners can use language-model apps, but few can explain — or implement — how tokenization, attention, training, and generation work together.”

Fragmented Tutorials

Tokenization, attention, and training are usually taught as disconnected pieces.

Research Code ≠ Engineering

The base repository works, but lacks tests, configuration logging, and design documentation.

The Gap

No clear, end-to-end, student-built and tested GPT pipeline with proper software documentation.

CHAPTER 1 · INTRODUCTION

Project Objectives

- 1 Study the Transformer/GPT architecture from foundational papers to nanoGPT.
- 2 Implement embeddings, masked multi-head attention, FFN, LayerNorm, and output head in PyTorch.
- 3 Build a GPT-2 BPE tokenized dataset pipeline with context windows.
- 4 Train & validate on a chosen corpus, logging loss, perplexity, and samples.
- 5 Implement temperature and top-k decoding strategies.
- 6 Refactor the repository into a clean, tested, reproducible codebase.
- 7 Produce a full set of UML/ERD/sequence/class design diagrams.

CHAPTER 3 · METHODOLOGY

Tools & Technology Stack

Python 3.10+

Core implementation language

PyTorch

Tensors, autograd, training

tiktoken

GPT-2 byte-pair tokenizer

Jupyter Notebook

Experiments & demos

pytest

Automated testing

Git & GitHub

Version control

VS Code

Development environment

CUDA GPU / CPU

Training compute

CHAPTER 3 · SYSTEM DESIGN

System Architecture

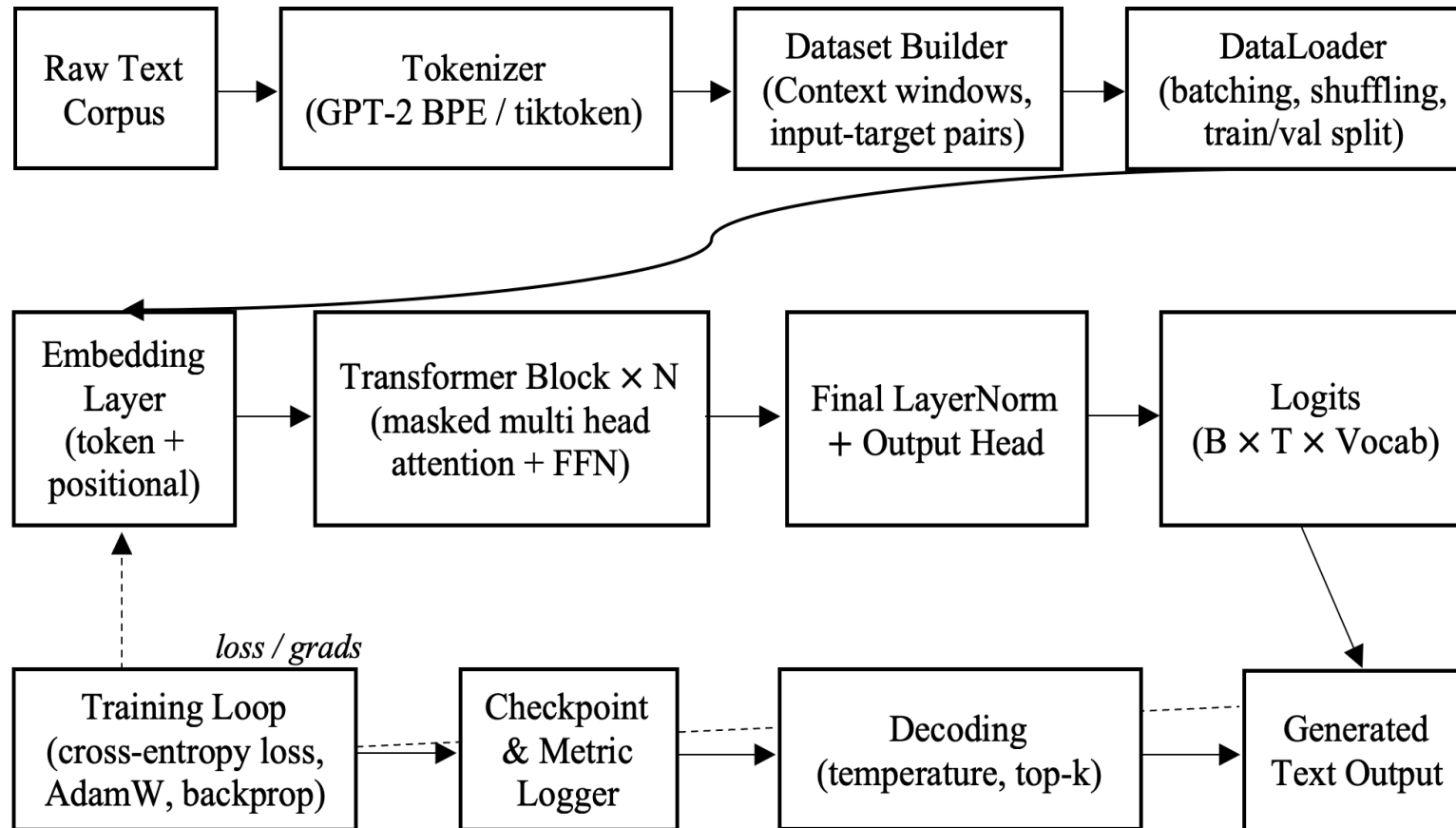


Figure 1: Proposed System Architecture

Raw text flows through tokenization, dataset windowing, batching, embeddings, a stack of transformer blocks, and an output head.

The training loop (bottom-left) updates weights from loss, while the decoding stage (bottom-right) produces generated text from the trained model.

CHAPTER 3 · SYSTEM DESIGN

Use Case Diagram

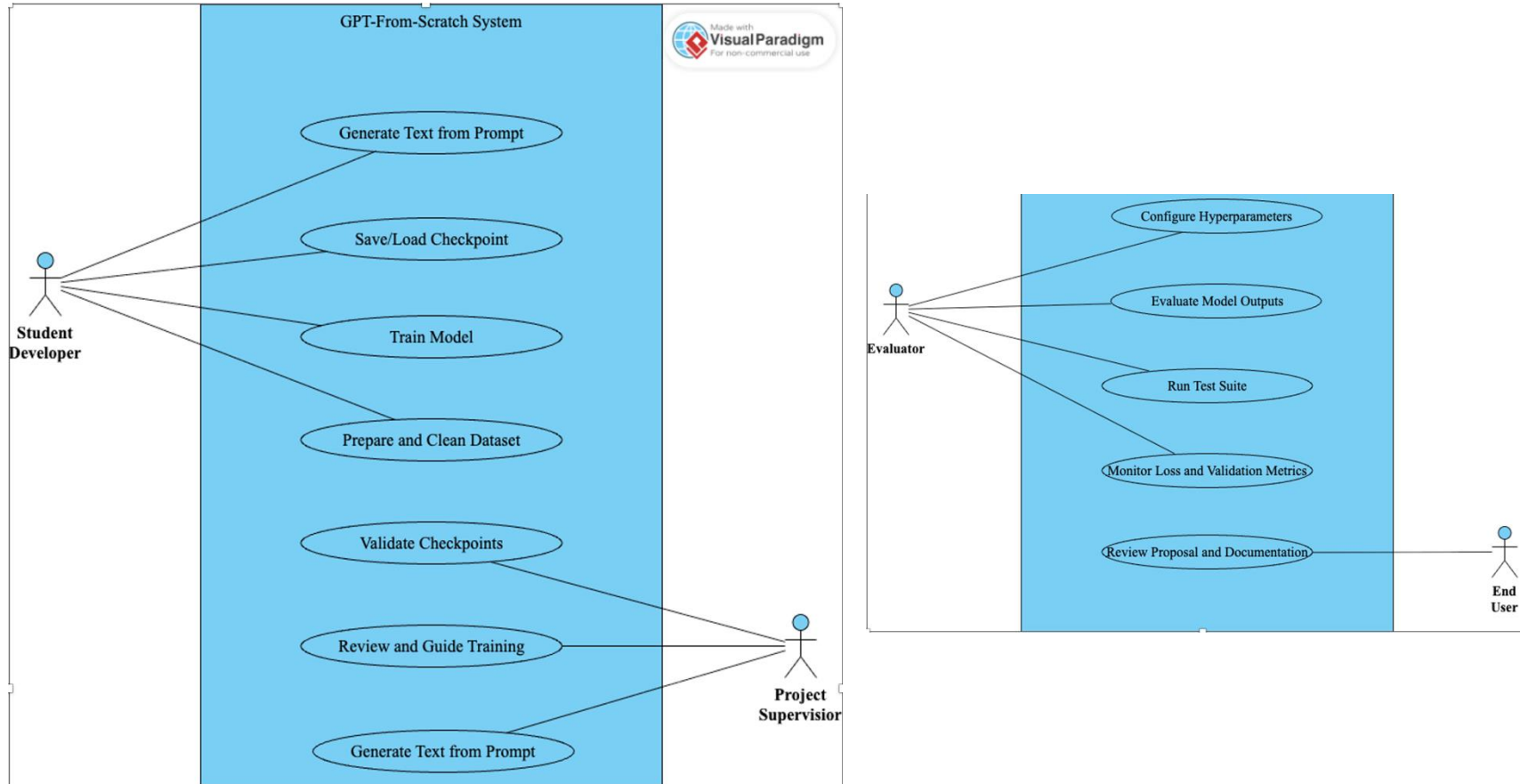


Figure 2: Use Case Diagram

Four actors interact with the system: the student developer, project supervisor, evaluator/panel, and end user.

Nine use cases span dataset prep, training, checkpointing, testing, generation, and documentation review.

CHAPTER 3 · SYSTEM DESIGN

ER Diagram – Experiment Tracking

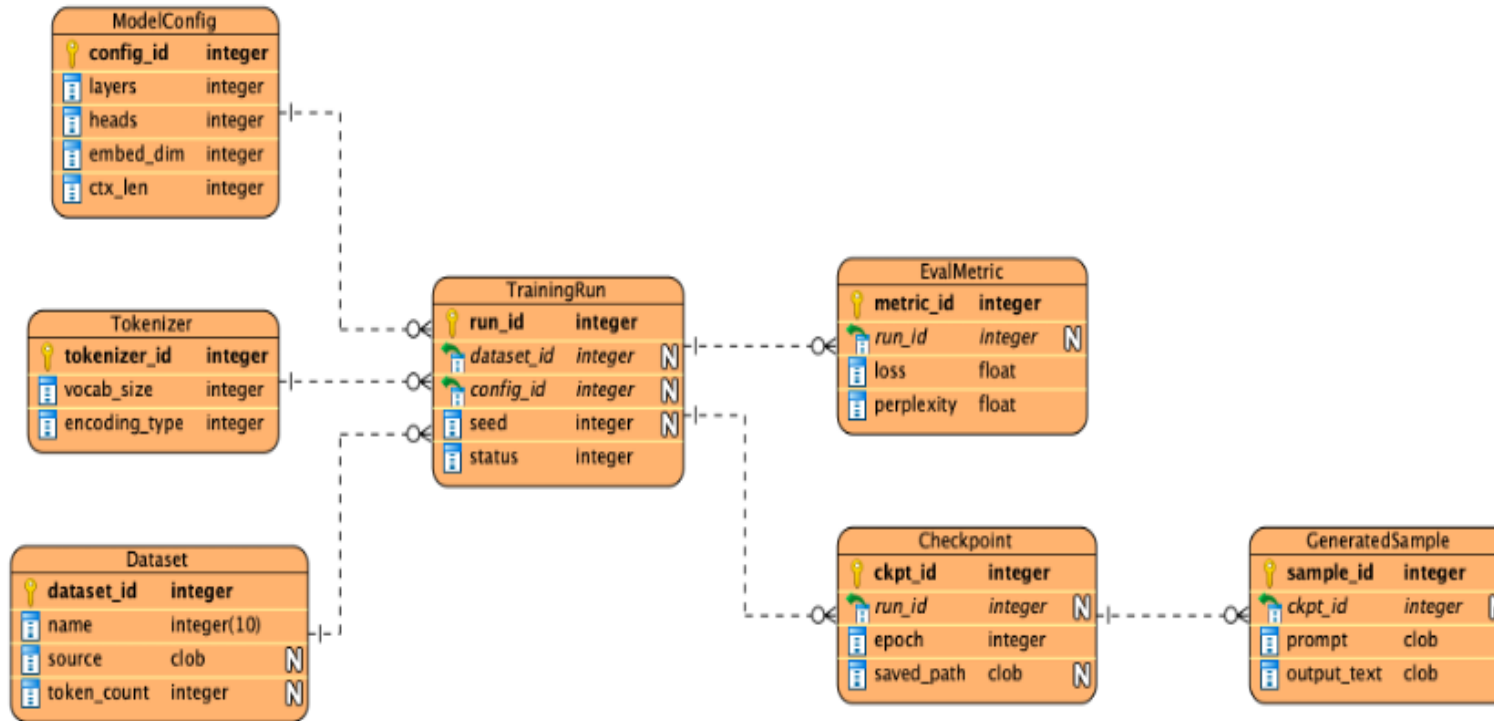


Figure 3: ER Diagram for Experiment Tracking

A lightweight schema (Dataset, Tokenizer, ModelConfig, TrainingRun, Checkpoint, EvaluationMetric, GeneratedSample) makes every result traceable to the exact configuration and data that produced it.

CHAPTER 3 · SYSTEM DESIGN

Sequence Diagram – Text Generation

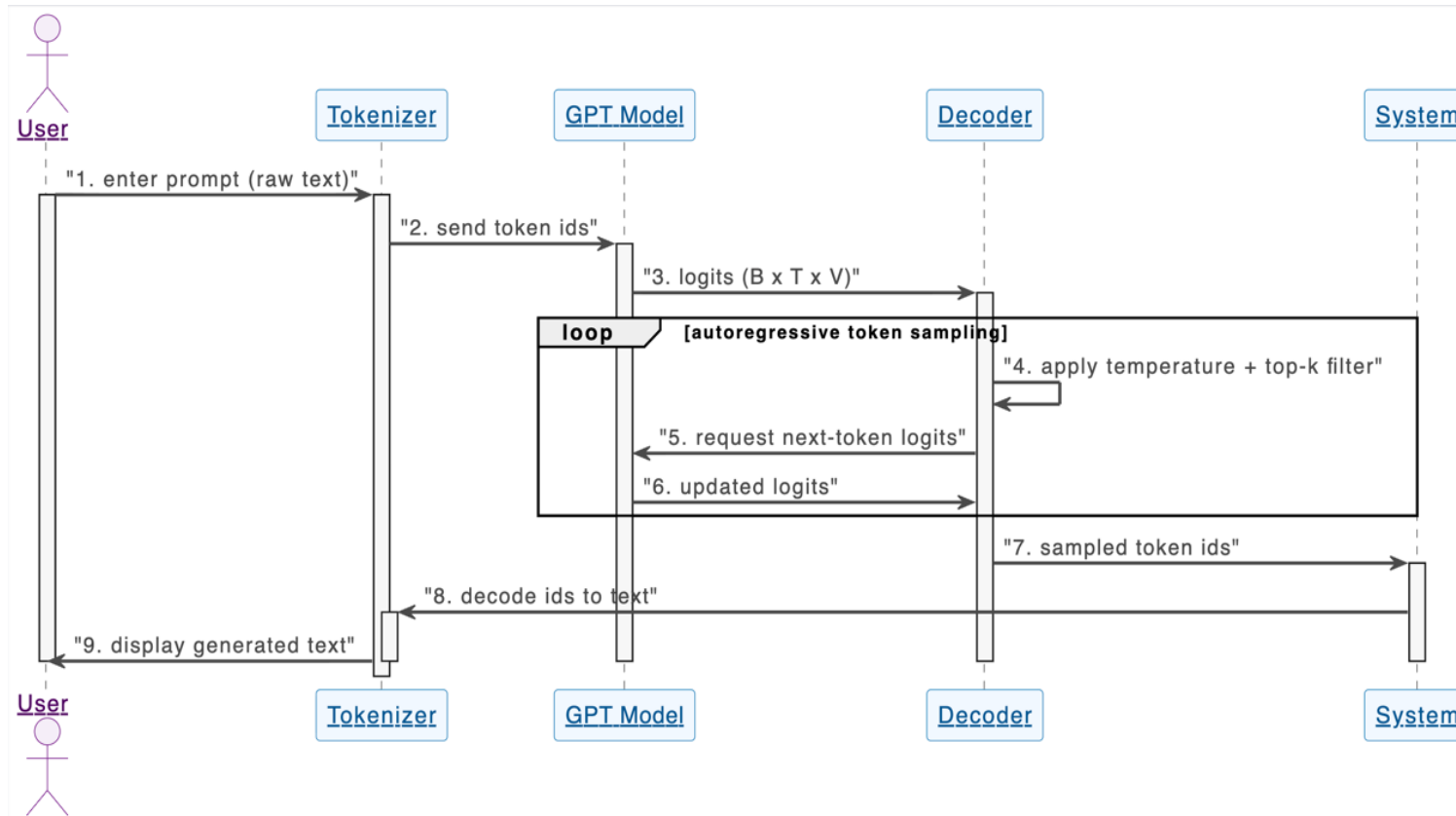


Figure 4: System Sequence Diagram – Text Generation

A prompt is tokenized, passed through the model, filtered by temperature and top-k, sampled autoregressively in a loop, then decoded back into readable text.

CHAPTER 3 · SYSTEM DESIGN

Sequence Diagram – Training Iteration

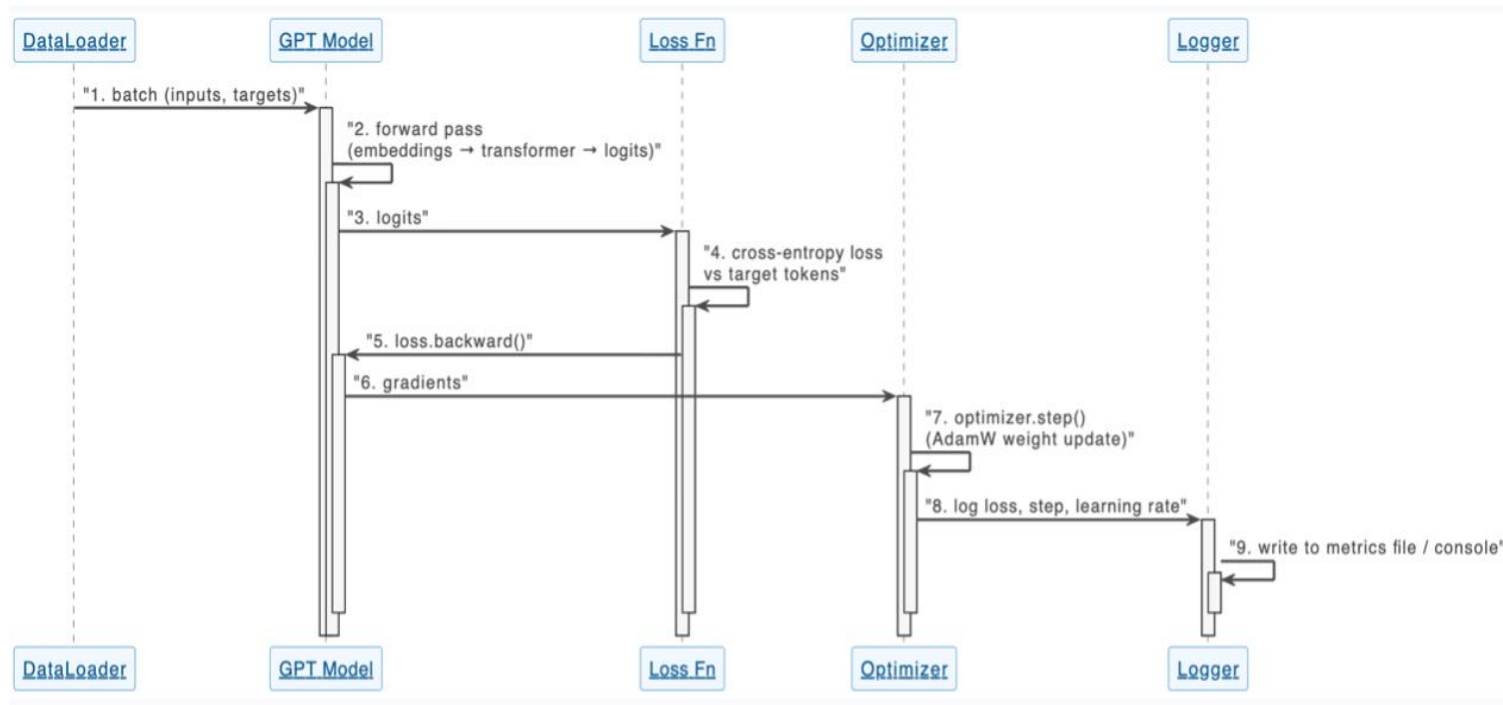


Figure 5: Sequence Diagram – One Training Iteration

One training step: batch → forward pass → cross-entropy loss → backward pass → AdamW update → metric logging.

CHAPTER 3 · SYSTEM DESIGN

Design Class Diagram

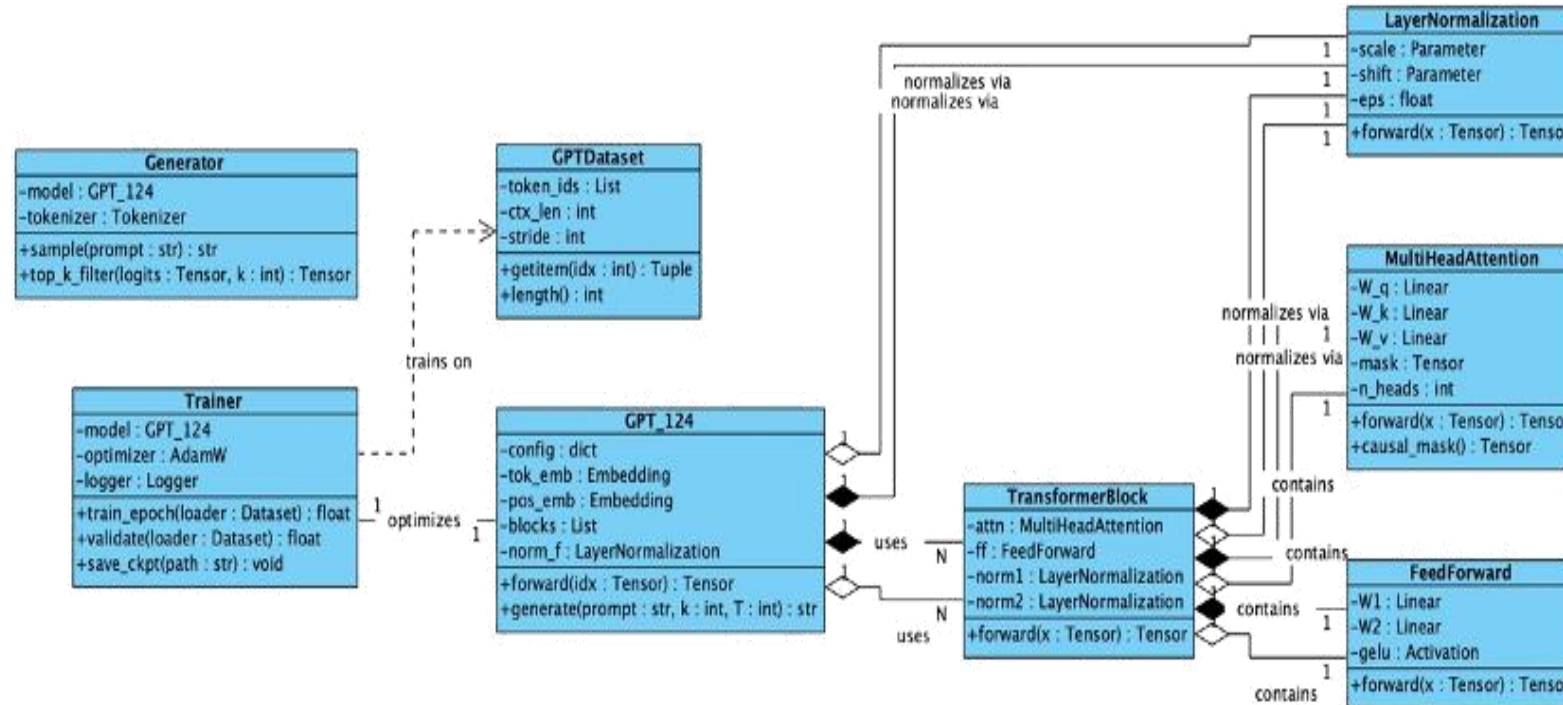
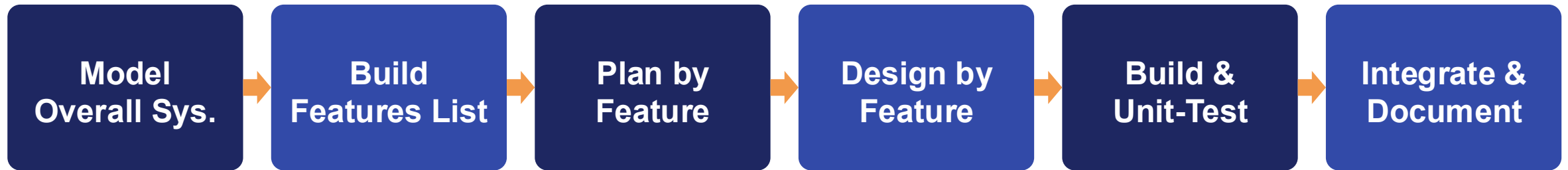


Figure 6: Design Class Diagram

GPT_124 composes a stack of TransformerBlocks, each containing MultiHeadAttention and FeedForward, normalized via LayerNormalization. Trainer and Generator wrap the training and inference workflows.

CHAPTER 3 · SYSTEM DESIGN

Feature Driven, Iterative Development



Each feature – tokenizer dataset, transformer block, training loop, evaluation, generation – is designed, implemented, unit-tested, and only then integrated. This keeps debugging local and produces a demonstrable increment every week.

- Tokenizer pipeline
- Dataset pipeline
- Transformer block
- GPT model
- Training loop
- Evaluation
- Generation
- Testing & docs

CHAPTER 3 · SYSTEM DESIGN

Testing Plan

Test Case	What It Verifies
Tokenizer Test	Encode/decode round-trip is correct
Dataset Shape Test	Tensor shapes match batch × context length
Attention Mask Test	No position attends to the future
Forward Pass Test	Output shape = batch × seq × vocab
Loss Test	Loss is finite, decreases on overfit sample
Generation Test	Output respects length & sampling limits
Checkpoint Test	Reloaded model reproduces identical logits

CHAPTER 3 · SYSTEM DESIGN

Risk Matrix and Mitigation

Impact	High	Compute / GPU limitation	Training instability	
	Medium	Overfitting on small corpus		Time Overrun near deadline
	Low	Checkpoint / data loss	Tokenizer edge class	
		Low	Medium	High
		Likelihood		

GPU/compute limits

Gradient accumulation, cloud fallback

Training instability

Grad clipping, LR warmup, frequent checkpoints

Overfitting

Validation split, dropout, monitoring

Time overrun

Weekly schedule, continuous documentation

Figure 8: Risk Matrix

CHAPTER 5 · SCHEDULE

8-Week Work Schedule

Tasks:

1. Requirement study and repository review
2. Literature review and dataset/tokenizer study
3. Dataset pipeline and data loader refactor
4. Model Implementation review and shape tests
5. Training loop, checkpointing, logging
6. Evaluation, decoding experiments, risk review
7. Unit testing and integration testing and Final report, presentation and demo preparation

Tasks	Weeks							
1	1							
2		2						
3			3					
4				4				
5					5 and 6			
6							7	
7								8

Figure 7: Project Gantt Chart

CHAPTER 4 · OUTCOME

Expected Outcomes

Working, end-to-end GPT-style model trainable on a chosen corpus

Documented, tested tokenizer & dataset pipeline

Training/validation logs showing visible learning

Generated samples under temperature & top-k sampling

Reproducible repo: config logging, checkpoints, tests, CLI

Complete proposal, mid-defence report, final report & slides

Thank You

Repository: github.com/beeks-code/GPT-From-scratch

Bikram Sharma • Pramesh Lamichhane • Swikar Poudel